

Vorlesung Cybersicherheit, Sommersemester 2025

Betriebssystemsicherheit mit Multics & Trusted Execution Environments

Prof. Dr.-Ing. Lucas Vincenzo Davi

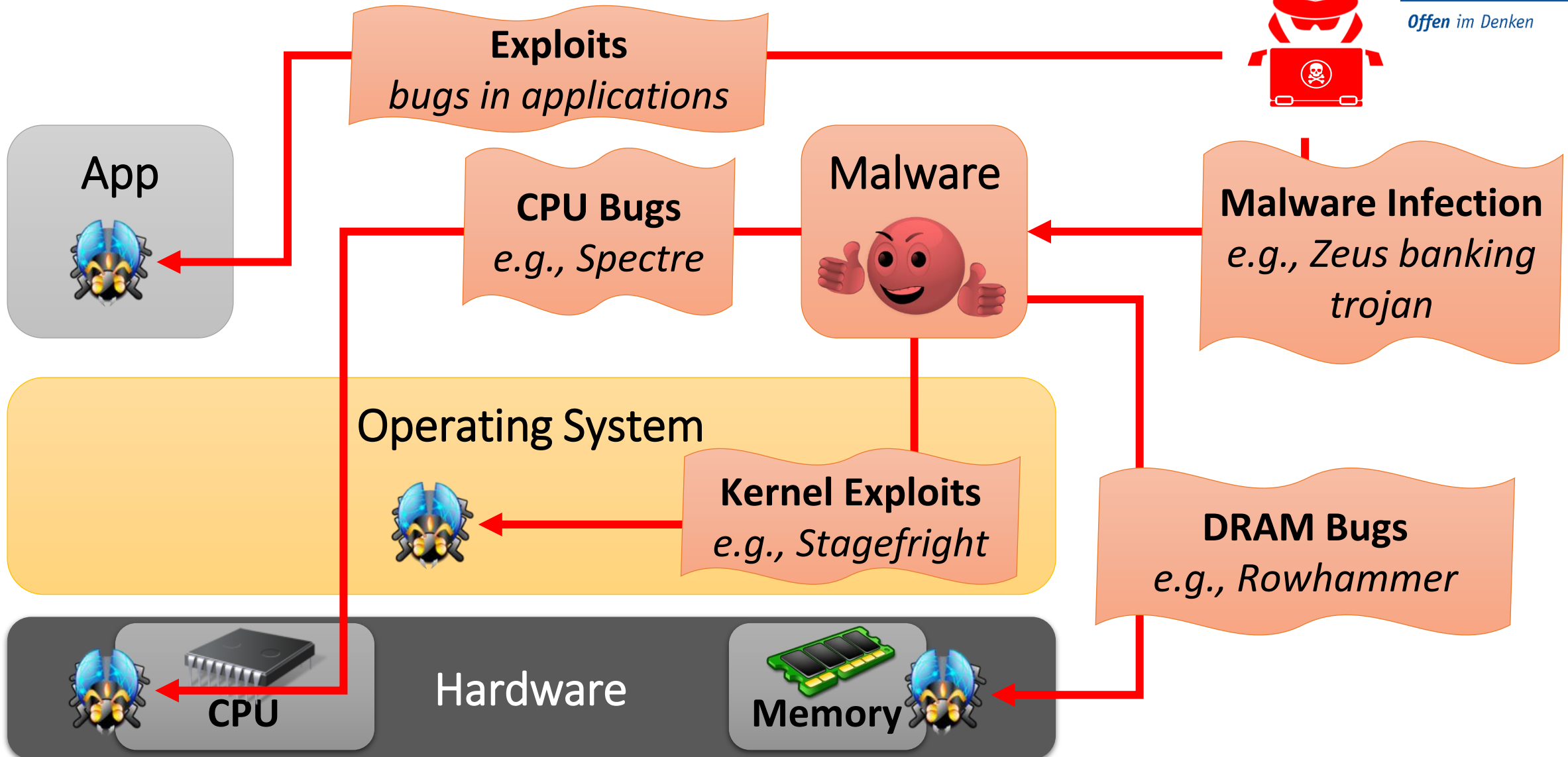
Fakultät für Informatik

Arbeitsgruppe Systemsicherheit <https://www.syssec.wiwi.uni-due.de/>

Universität Duisburg-Essen

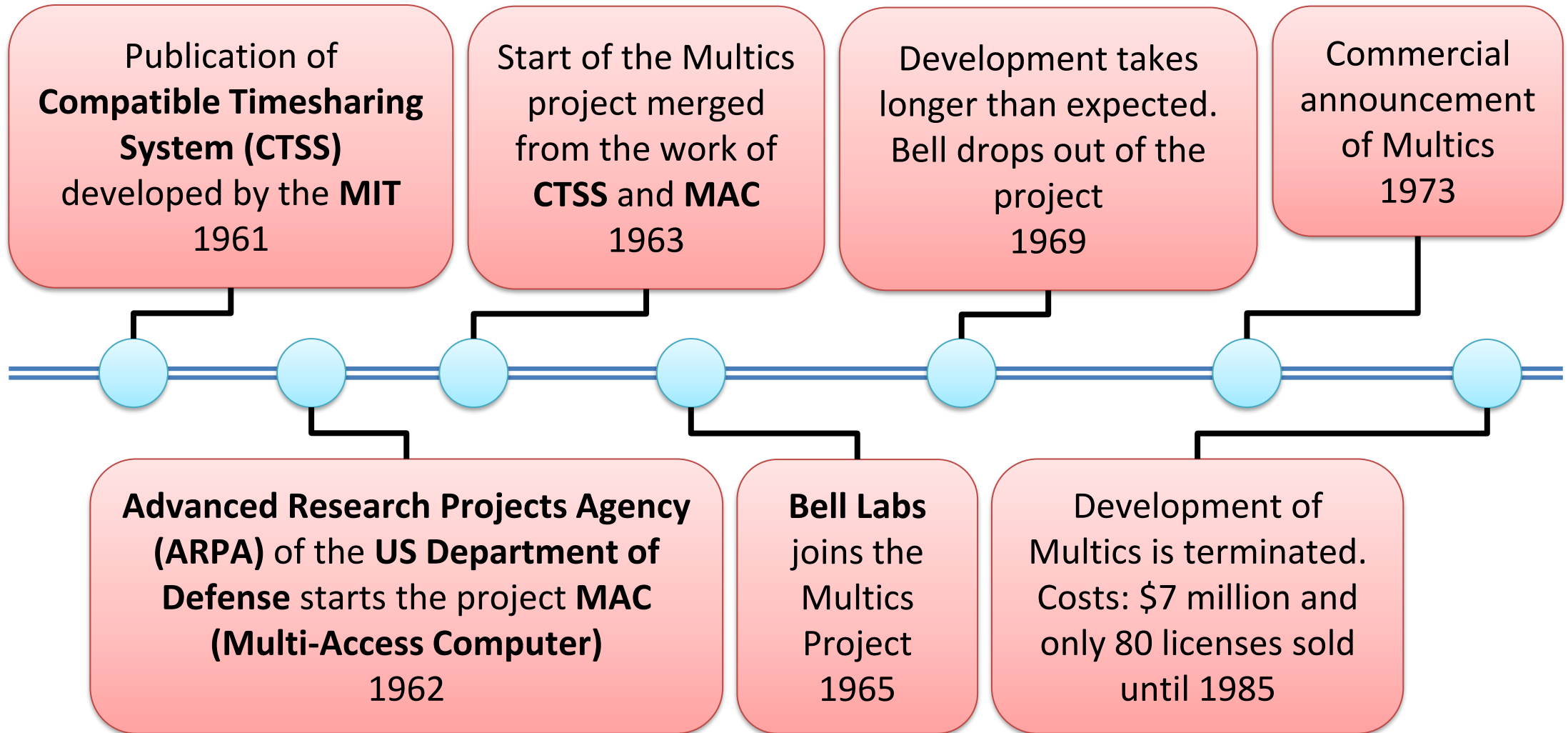
© SYSSEC, Prof. Dr. Lucas Davi

The Threat Landscape



- In this chapter we introduce and examine the secure operating system **Multics** (**Multiplexed Information and Computing Service**)
 - Multics was the first modern operating system
 - The Multics project introduced many operating system concepts that are widely used in operating systems today
 - Virtual memory
 - Hierarchical file system
- Moreover, fundamental **secure** operating system concepts were invented and implemented in Multics
 - Reference monitor
 - Protection rings
 - Multilevel security policies

History



Processes and Segments

- The fundamental concepts in Multics are **processes** and **segments**
- Segments are organized in a hierarchy of **directories**

Process

All executable content in Multics
(i.e., they run program code)

Each process has its own virtual
address space in memory

A process is divided into
segments

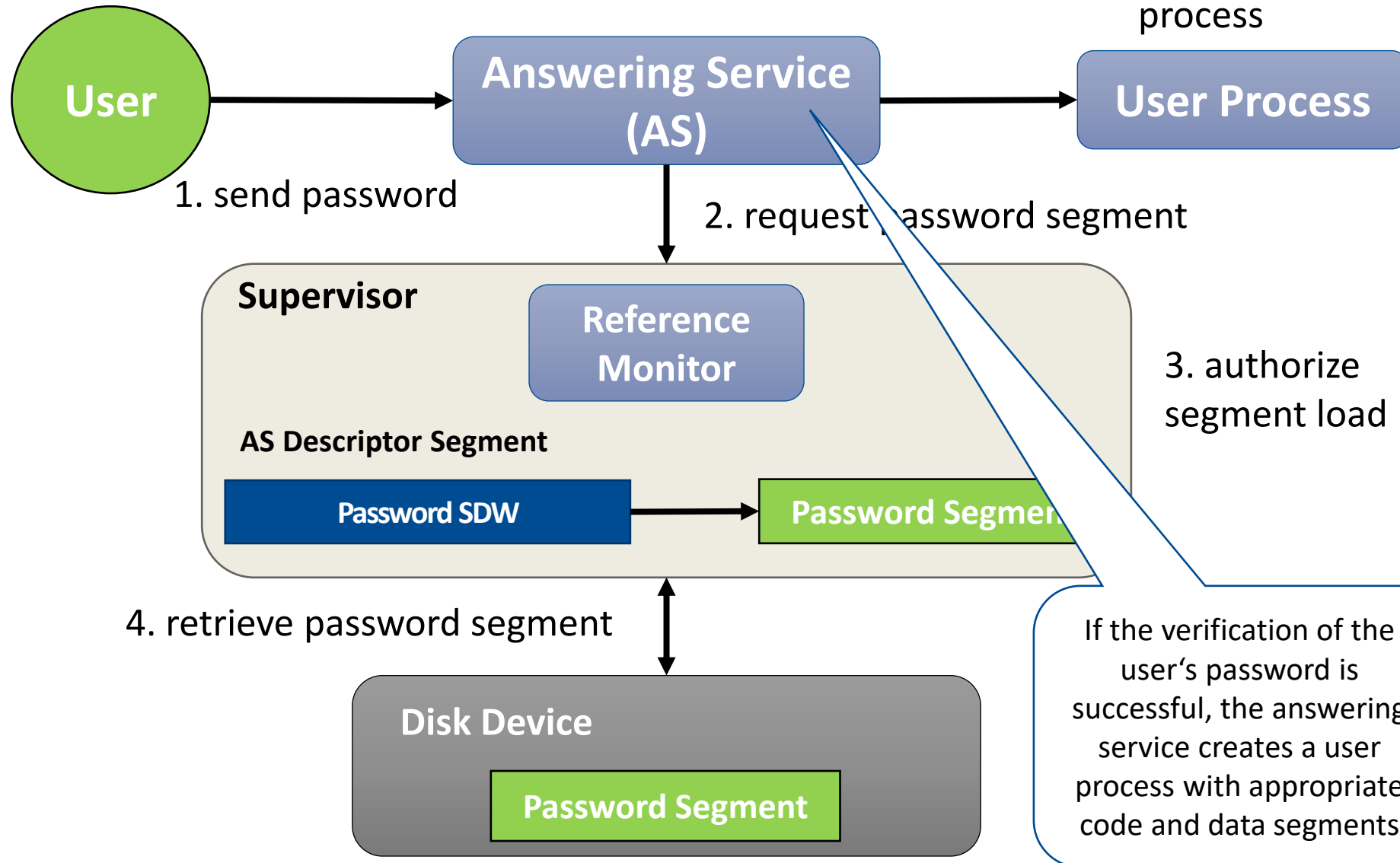
Segment

Subdivision of a process address
space

All resources (e.g., code, data,
I/O devices etc.) that may be
accessed by a process

- Segments are addressable either **locally** or by their name **from secondary storage** (e.g., from disk)
- Each process holds its own **descriptor segment** for locally addressable segments
 - The descriptor segment contains descriptor segment words (SDWs)
 - Each SDW points to a directly addressable segment in system memory
 - Loading SDWs into the process descriptor segment is performed by the Multics **supervisor**

Multics Login Process



The Supervisor and the Answering Service

The answering service

- Responsible for verifying the user's password
- To verify the password, the answering service needs the password segment from the file system
- The loading of the password SDW to the answering service's descriptor segment must be **authorized and enforced by the supervisor**

The Multics supervisor

- The supervisor implements the most trusted components of the Multics system
- Only the supervisor segments are assigned to ring 0 and ring 1

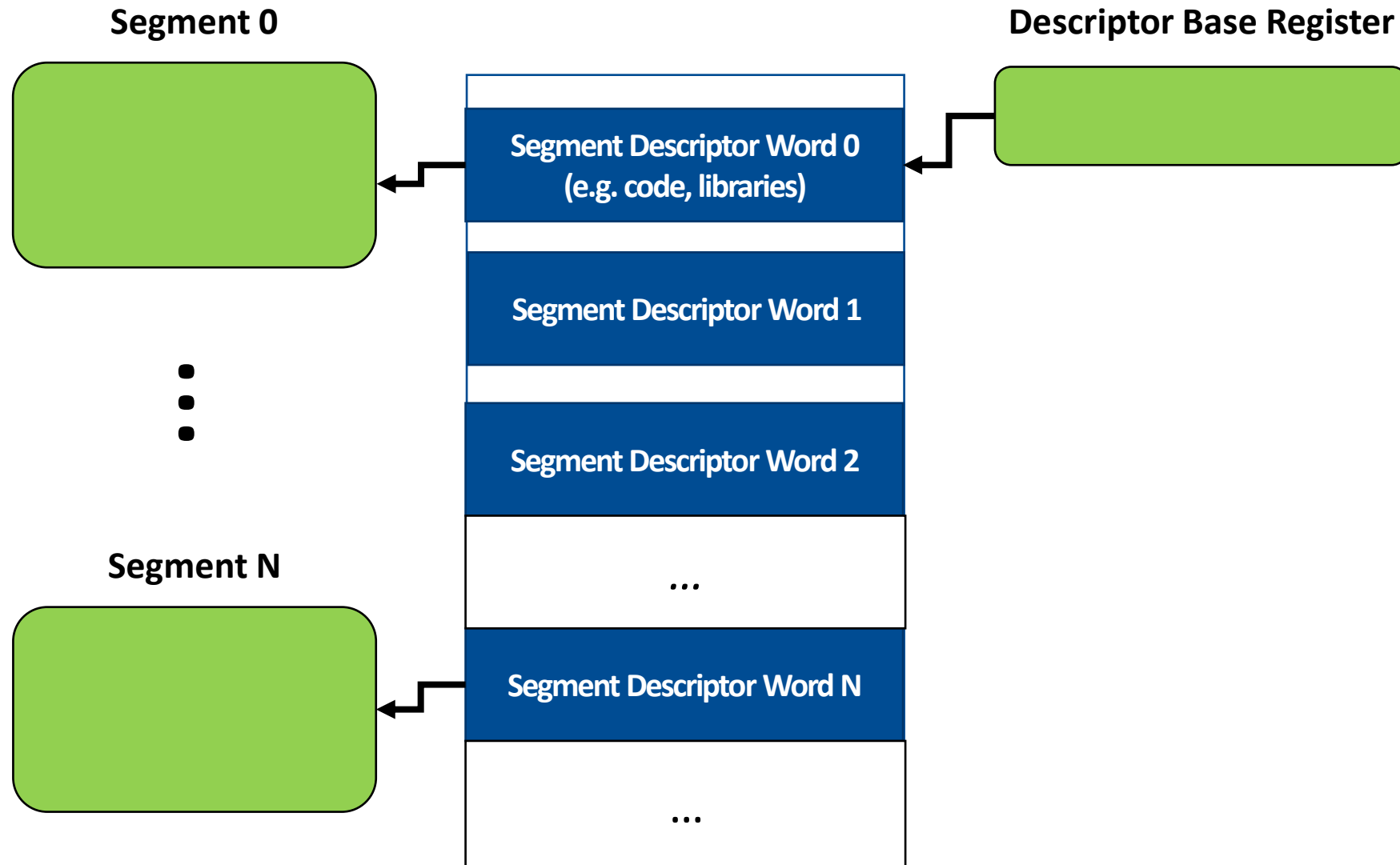
Notion of Reference monitor

- Combination of hardware and software responsible for enforcing the security policy of the system

Reference monitor principles

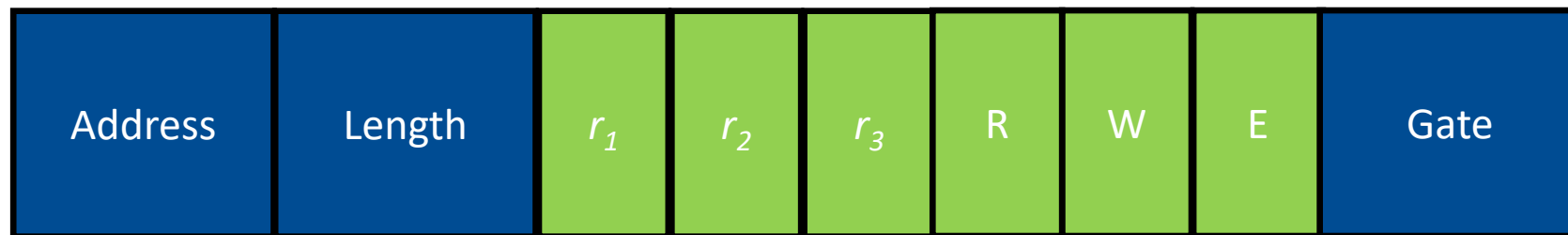
1. **Complete Mediation**: Any access to information must be mediated by the reference monitor. It must be impossible to bypass the reference monitor.
2. **Tamperproof** (Isolation): The system must protect the reference monitor, including its protection system from modification
3. **Verifiable**: The reference monitor must be small enough to be subject to analysis and test in order to prove that the system enforces its security goals

Process Descriptor Segment in Multics



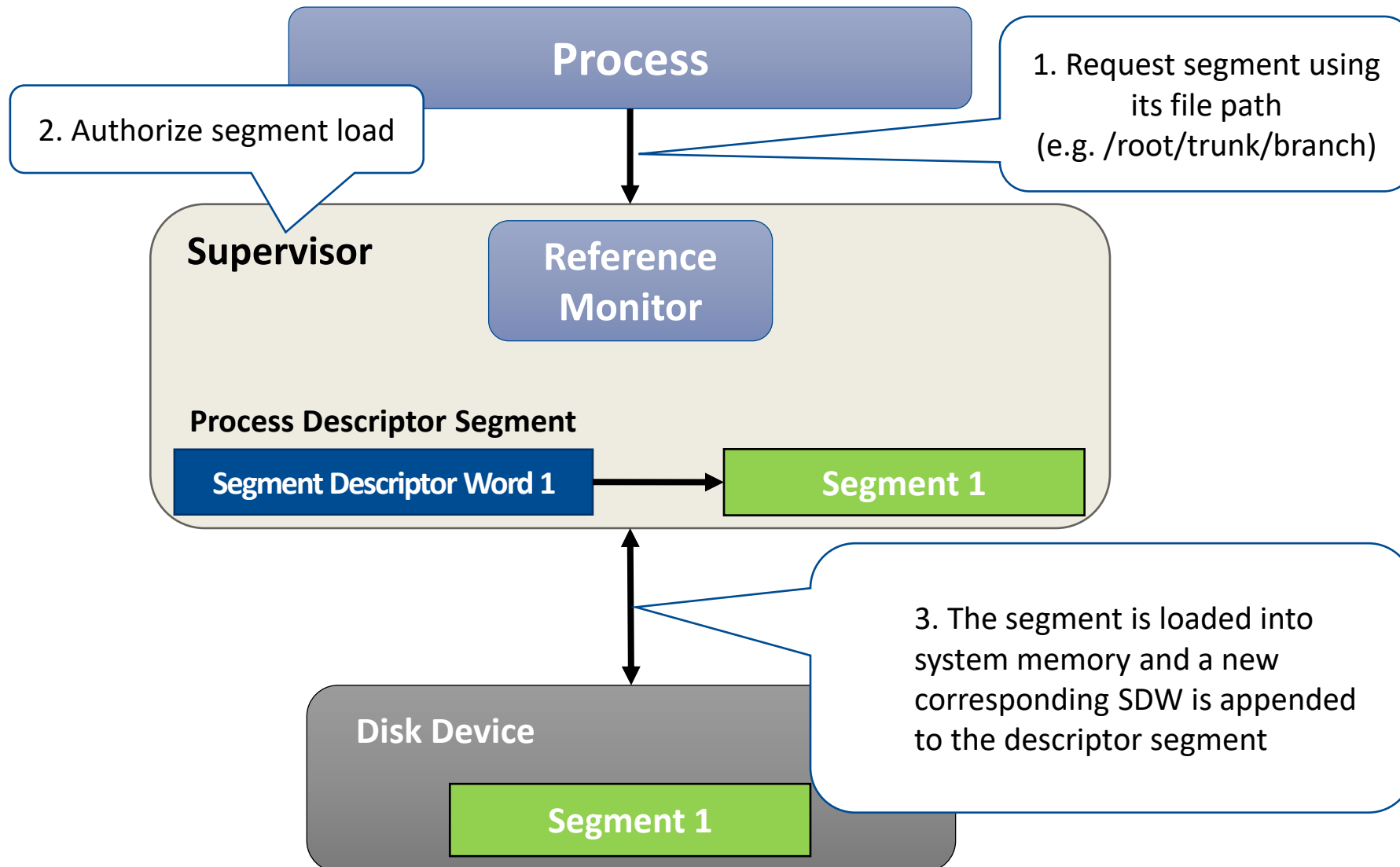
The Segment Descriptor Word (SDW)

- The SDW contains the following fields
 1. The segment's address in memory
 2. The length of the segment
 3. Access indicators
 - Ring Brackets (r_1, r_2, r_3) used for the protection ring model
 - The process permissions for the segment (Read, Write, Execute) used for Access Control Lists
 - Gate: A list of valid entry points to higher-privileged rings. This field is used by gate segments and is needed in the protection ring model for ring transitions.



- Notion of **code segment**
 - Segments that contain code are referred to as **code segments** (i.e., the segment is executable)
- Notion of gate segment
 - A **gate segment** is a special kind of code segment that uses the **gate** field in the SDW to provide valid entry points to higher-privileged protection rings
 - If a process invokes code that needs to run in a higher-privileged protection ring, then the process has to enter the higher-privileged protection ring through a gate segment
 - Therefore, a gate segment provides a **list of valid entry points** (gate field) and the process must start execution at one of these entry points
 - At these entry points **input arguments are validated** in order to protect the invoked code from malicious input originating from lower-privileged protection rings

Process Requesting a Segment



Notion of Protection Rings

- Protection rings form a **hierarchical layering** from the most privileged ring (ring 0) to the least privileged ring

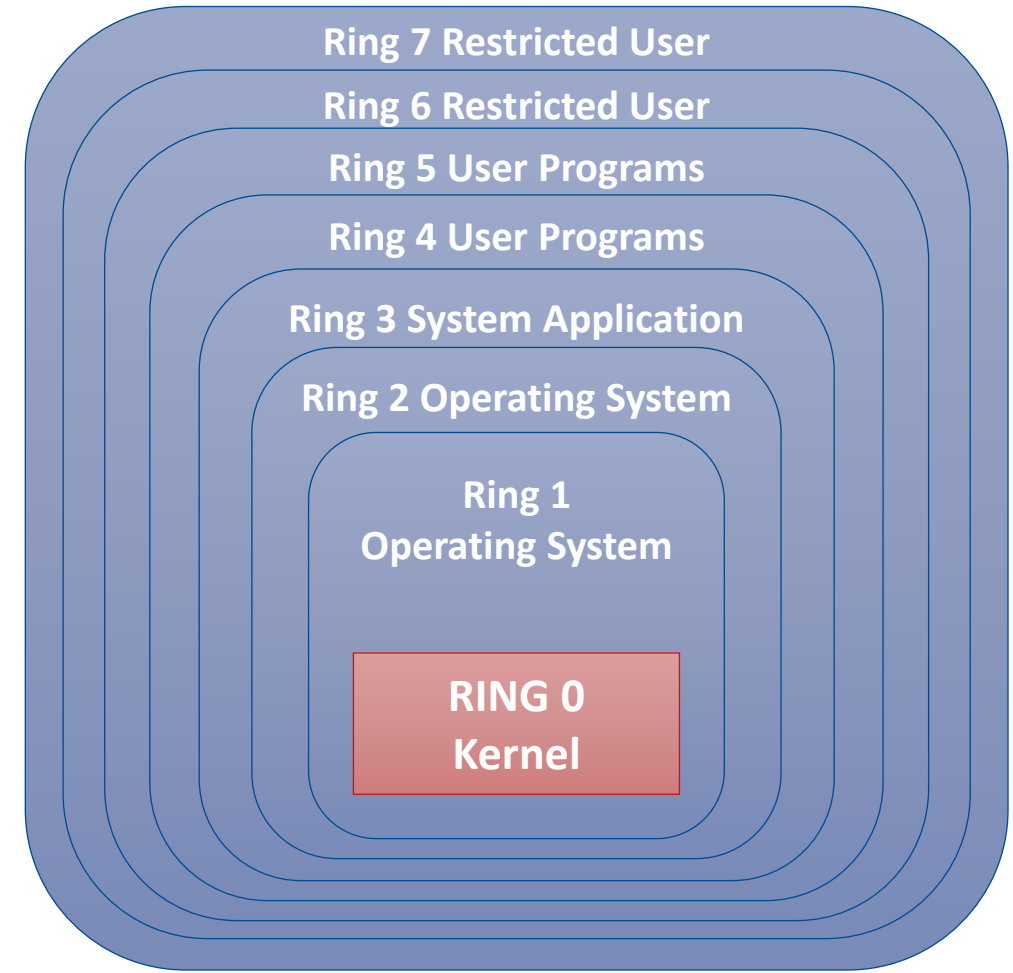
Protection rings in Multics

- Multics provides 64 protection rings
 - 8 protection rings were implemented in the hardware
 - The remaining 56 protection rings were implemented in software
- The Multics **supervisor** assigns the ring numbers to **processes** by using a special processor register¹ that holds the current ring number of a process
- Ring numbers of **segments** are stored directly in SDWs

¹Effectively, the IPR register (instruction pointer) specifies the current ring of execution

General Example on Protection Rings

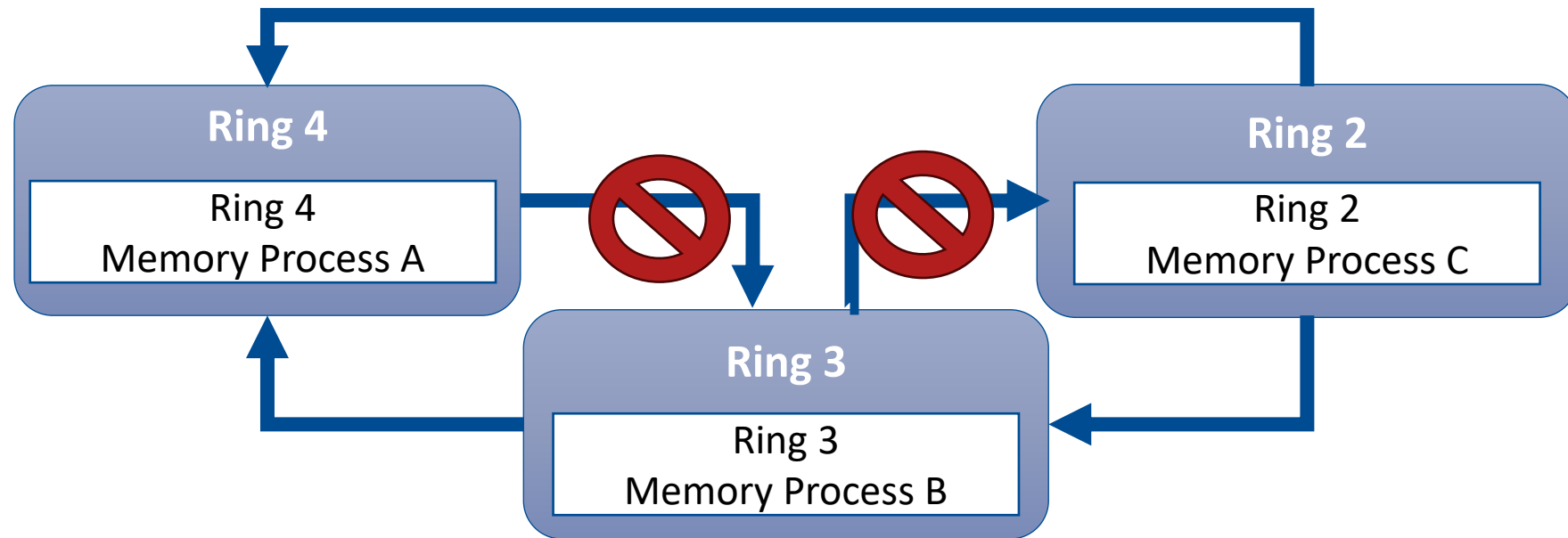
- The rings of privilege show the **most privileged ring in the center** with less privileged rings as we move outward
- The **innermost rings** are occupied by the operating system
- In **intermediate rings** users are doing their programming
- The **outermost rings** can be assigned to restricted users



Memory Access in Protection Rings

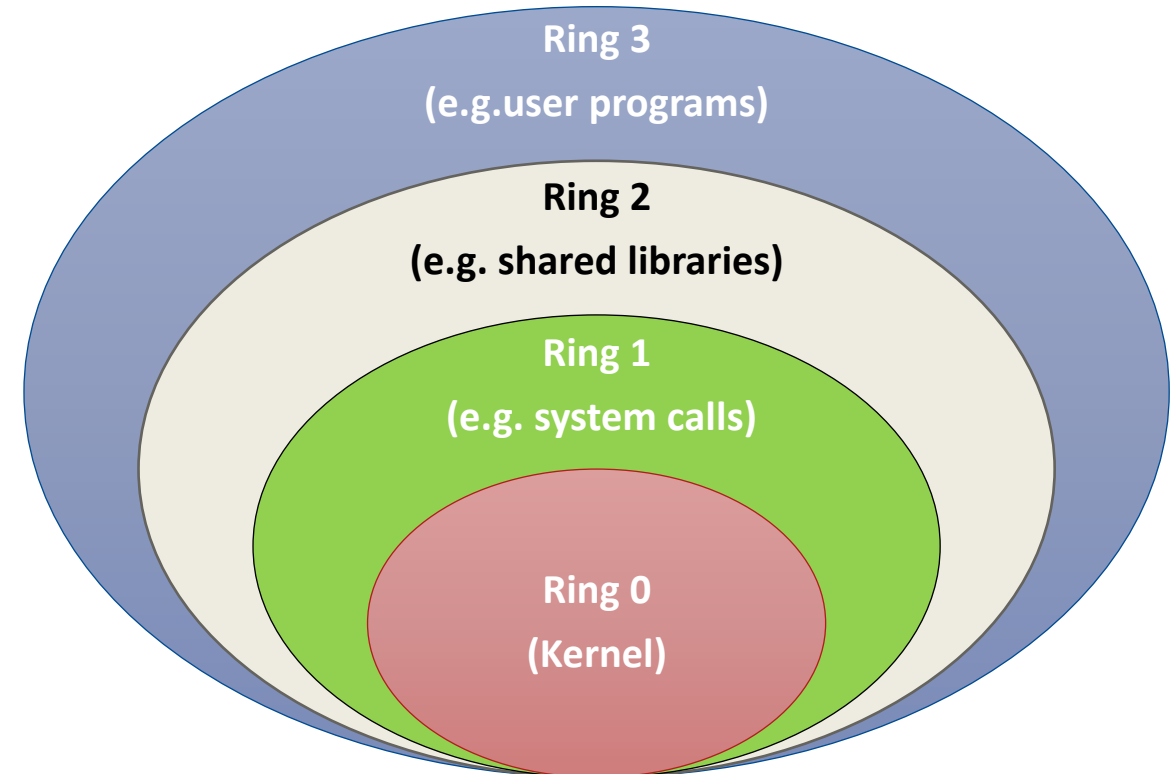
Higher-privileged rings have **at least access to all memory of lower-privileged rings**

- Processes in higher-privileged rings tend to share memory across all processes
- Processes (e.g., user processes) in lower-privileged rings are isolated from one another



Protection Rings in Modern Systems

- Intel provides 4 protection rings
- However, modern operating systems (e.g., Windows and Linux) **only use ring 0 and ring 3**, because some hardware architectures of the past only supported two privilege levels



Permission Checks in Multics

- The Multics protection system consists of the following three models
 - Access Control List (ACL)
 - Rings and Brackets
 - Multilevel Security
- If a process requests access to a segment **all three policies/models must authorize the request**
- Policy Classification
 - Discretionary Access Control (DAC): At the discretion of the owner
 - ACL, Rings and Brackets
 - Mandatory Access Control (MAC): Immutable system-wide policies instantiated at boot
 - Multilevel Security

ACL in the Context of Multics

- Basically, a list of users and their access permissions
- Each segment and directory is associated with its own access control list (ACL)
- Each ACL entry defines a **user identity of processes** and the permissions that processes running on behalf of that user have

- The full user ID is a three-component name: **Person.Project.Tag**
 1. **Person**: The name of the user
 2. **Project**: The name of group the user is working in
 3. **Tag**: Indicates if the user is using the system interactively (m) or as an “absentee (batch)” user (a)
- Example *Jim.ProjectX.**
 - The user Jim is working within the group *ProjectX* and uses the system both interactively and as an absentee user
 - The wildcard (*) means that any person/project/tag is acceptable

Permission modes for segments

- read (r)
- write (w)
- execute (e)
- null (n)

Permission modes for directories

- Status (s): List the contents of a directory
- Modify (m): Delete objects and modify attributes of objects from a directory
- Append (a): Add a new object to a directory

A process gains access to a segment, if the associated user has an entry in the ACL that permits the requested operation

Example on ACLs for Segments

- ACL for the file */projects/X/accounting*

Permissions	User	Description
n	Tom.*.*	Tom has no access to the file
rwe	Jim.ProjectX.*	Jim working for ProjectX has read/write/execute permissions
r	Backup.SysDaemon.*	The Backup Daemon can read the file
rw	*.SysAdmin.a	Batch users of the SysAdmin group are able to read and write to the file

Example on ACLs for Segments

- ACL for the directory */projects/X/*

Permissions	User	Description
sma	Jim.ProjectX.*	Jim working for ProjectX has status/modify/append permissions
s	Backup.SysDaemon.*	The Backup daemon can list the contents of the directory
sm	*.SysAdmin.m	Interactive users of the SysAdmin group have status and modify permissions

Motivation for Protection Rings

- A descriptor segment without ring brackets defines a single **domain of execution**
 - A set of access capabilities associated with a process is referred to as **domain**
 - The procedures executed as part of a process have all the same access capabilities

However, sometimes it is necessary to change the process's access capabilities during program execution

- A user program invokes a supervisor procedure (e.g., a system call)
 - starting an I/O operation
 - or changing the access control list of a segment
- The system call needs more privileges (in order to execute successfully) than the user process has
- Thus, for the time the system call executes, the user process needs extended privileges

Associating multiple domains with a single user process is accomplished in Multics by **protection rings**

Basic Principles of Rings and Brackets

- Protection rings for a segment are stored in the SDW
 - Each segment is associated with a ring bracket



- Depending on the process's current ring (r) and the ring bracket (r_1, r_2, r_3) of the segment, access is permitted or denied

Basic Principles of Rings and Brackets

- **Access Brackets:** Defines ranges of rings that are allowed to **read** and **write** segments (i.e., processes whose current ring number is within the defined range are allowed to read/write to the segment)
- **Call Brackets:** Defines ranges of rings that are allowed to **execute** code

- Access Bracket: (r_1, r_2)
 - r_1 : The highest ring which can **write** to the segment
 - r_2 : The highest ring which can **read** the segment
 - Always $r_1 \leq r_2$: r_1 is more privileged than r_2
- Possible relations (current protection ring is r)
 - $r \leq r_1$: The process can read and write to the segment
 - $r_1 < r \leq r_2$: The process can read the segment, but is not allowed to write to the segment
 - $r_2 < r$: The process can neither read nor write to the segment

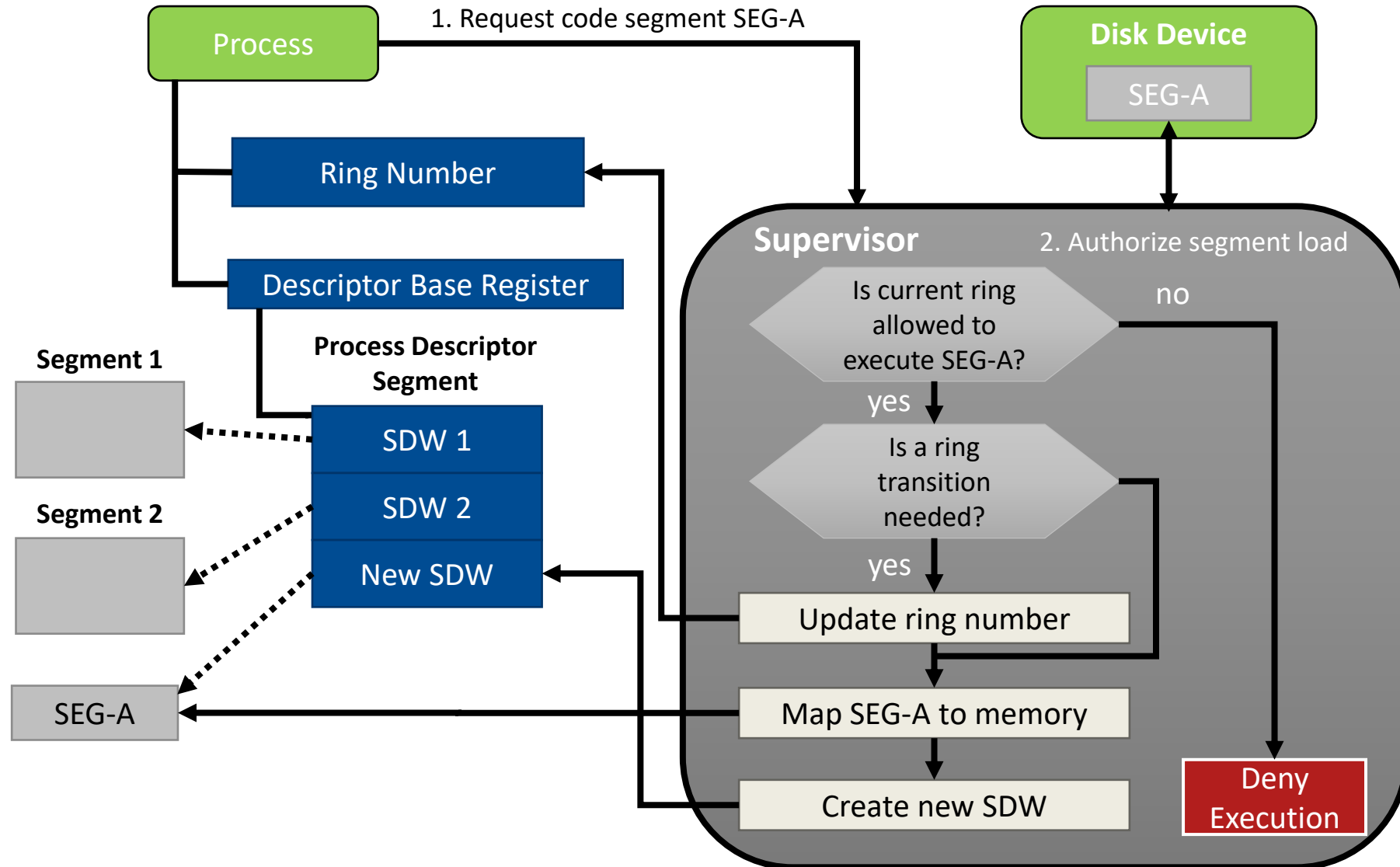
Example for Access Brackets

- Examples (current protection ring is $r = 4$)
 - (5, 6): Process is allowed to read and write to the segment
 - Because $r < r_1 \leq r_2$
 - (3, 4): Process is allowed to read, but is **not** allowed to write to the segment
 - Because $r_1 < r \leq r_2$
 - (3, 3): Process has no access to the segment
 - Because $r_1 \leq r_2 < r$

- Call Bracket: (r_2, r_3)
 - r_2 : The highest ring which can **read** the segment
 - r_3 : The highest ring which can **execute** code of the segment
 - For gate segments: $r_2 < r_3$: r_2 is more privileged than r_3
 - For all **non-gate segments**: $r_2 = r_3$
 - According to the transition rules defined in the following, a ring transition to a higher-privileged ring can only be enforced if $r_2 < r_3$
 - To identify if a code segment is a gate segment, it is sufficient to examine the call bracket: if $r_2 < r_3$ then the segment is a gate segment

- Ring transitions occur when a code segment belonging to a different ring is invoked by a process
- Two possible ring transitions
 1. Transition to a higher-privileged ring
 2. Transition to a lower-privileged ring
- A ring transition to a higher-privileged ring requires entry through a **gate segment** that validates input arguments originating from lower-privileged rings
 - Argument validation in Multics is quite sophisticated; it features explicit and implicit pointer routines

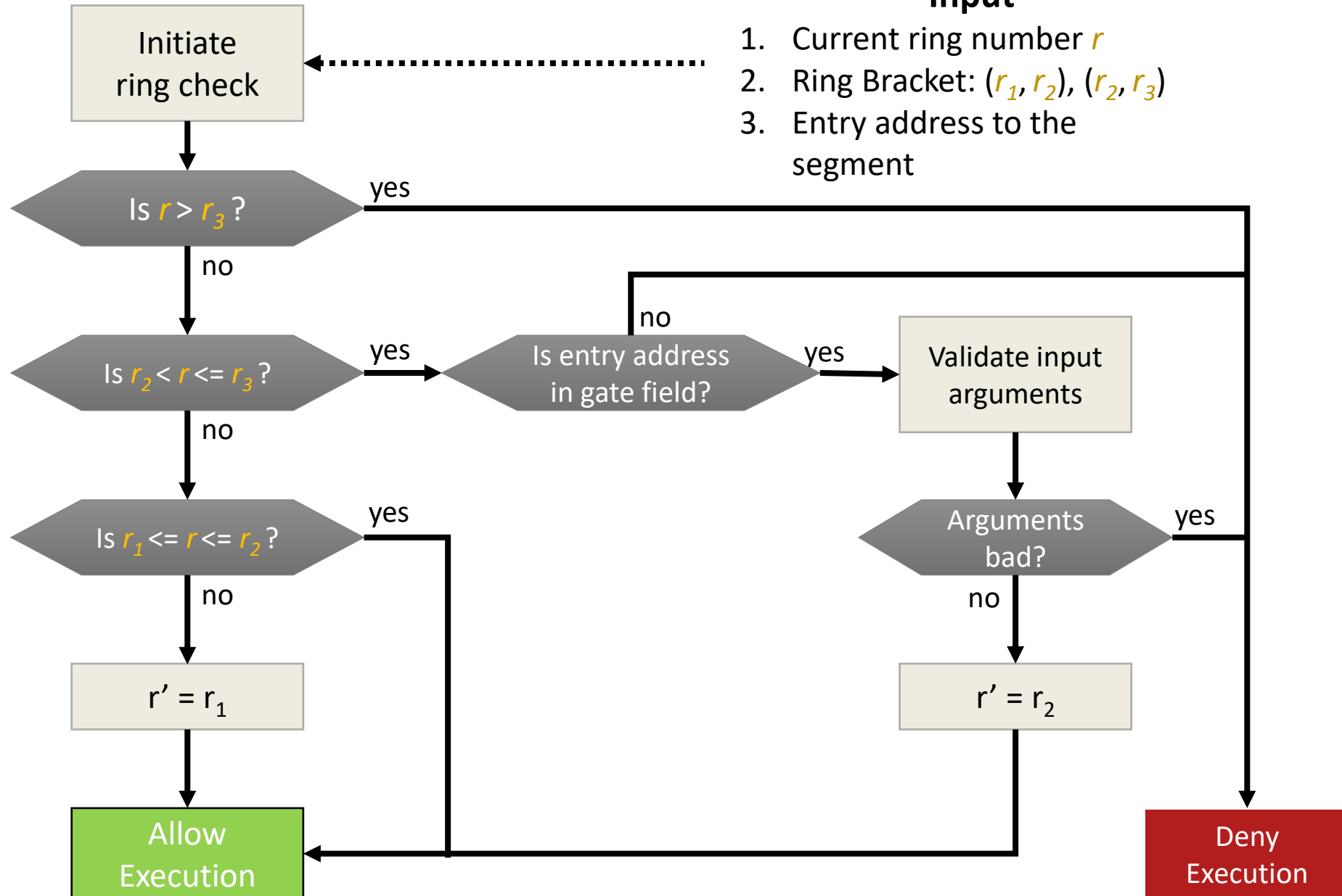
Ring Bracket Check on Code Segments



Flowchart for Ring Bracket Check

Input

1. Current ring number r
2. Ring Bracket: $(r_1, r_2), (r_2, r_3)$
3. Entry address to the segment



The Supervisor's Permission Check

- We now examine the possible results of the supervisor's permission check depicted on the last slide
- A process running in ring r requests invocation of a code segment with an access bracket (r_1, r_2) and a call bracket (r_2, r_3)

Cases	Description
$r_3 < r$	• Execution not allowed
$r_2 < r \leq r_3$	• Execution allowed through a gate • Ring transition to the higher-privileged r_2
$r_1 \leq r \leq r_2$	• Execution allowed • No ring transition
$r < r_1$	• Execution allowed • Ring transition to the lower-privileged ring r_1

Case: $r_3 < r$

- The process is not allowed to execute the code segment
 - The process's current ring number is not within the range of rings $(0, r_3)$ allowed to execute the code segment
- Examples
 1. $r = 6$, access bracket $(0, 0)$, call bracket $(0, 5)$
 - The process requests invocation of a gate segment ($r_2 < r_3$)
 - Since $r > r_3$ ($6 > 5$) execution of the gate segment is denied
 2. $r = 4$, access bracket $(1, 3)$, call bracket $(3, 3)$
 - The process requests invocation of a non-gate segment ($r_2 = r_3$)
 - Since $r > r_3$ ($4 > 3$) execution of the code segment is denied

Case: $r_2 < r \leq r_3$

- Description
 - The process requests a code segment of a higher-privileged ring
 - This case only occurs if the code segment is a gate segment ($r_2 < r_3$)
 - A popular use case for such ring transitions are **system calls**
- **Under certain conditions**, the process is allowed to execute the gate segment
 - **Condition 1**: Entry address to the segment must be defined in the gate field of the SDW
 - **Condition 2**: Argument validation is successful
- Ring transition rule 1
 - If $r_2 < r \leq r_3$ the process's current execution ring r is changed to r' , whereas $r' = r_2$

Case: $r_2 < r \leq r_3$

- Examples
 - We assume that argument validation was successful and that the process provided a valid entry address to the segment
 1. $r = 4$, access bracket $(0, 2)$, call bracket $(2, 6)$
 - According to ring transition rule 1, a ring transition to $r' = r_2 = 2$ is enforced
 - Privileges of the process are **enhanced** during the time the gate procedure executes
 - Upon return of the gate procedure, the execution is resumed to ring 4
 2. $r = 4$, access bracket $(3, 3)$, call bracket $(3, 4)$
 - Ring transition to $r' = r_2 = 3$

Case: $r_1 \leq r \leq r_2$

- The process is allowed to execute the code segment
 - First, the process's current ring number is within the range of rings $(0, r_3)$ allowed to execute the code segment
 - Second, the process's current ring number is within the following range of rings: (r_1, r_2)
 - This is the only case where the process can execute the code segment **without changing its ring of execution**
- Examples
 1. $r = 2$, access bracket $(1, 3)$, call bracket $(3, 3)$
 - The process requests invocation of a non-gate segment ($r_2 = r_3$)
 - The code segment is executed in ring 2
 2. $r = 2$, access bracket $(1, 3)$, call bracket $(3, 5)$
 - The process requests invocation of a gate segment ($r_2 < r_3$)
 - The gate segment is executed in ring 2

Case: $r < r_1$

- The process is allowed to execute the code segment
 - The process requests invocation of a code segment belonging to a lower-privileged ring
- Ring transition rule 2
 - If $r < r_1$ the current execution ring of the process r is changed to r' , whereas $r' = r_1$
- Examples
 1. $r = 0$, access bracket (1, 1), call bracket (1, 1)
 - According to ring transition rule 2, a ring transition to $r' = r_1 = 1$ is enforced
 - Privileges of the process are **decreased** during the time the code executes
 - Upon return of the invoked code, the process's execution is resumed to ring 0
 2. $r = 0$, access bracket (3, 5), call bracket (5, 5)
 - Ring transition to $r' = r_1 = 3$

- Multics pioneered the enforcement of Multilevel Security
 - Each directory stores a mapping from each segment to its secrecy level
 - Further, Multics stores a mapping from each process to its secrecy level
 - The MLS policy enforced by Multics realizes the Bell-La Padula (BLP) model and prevents **information leakage**
 - Typical secrecy levels: top-secret, secret, confidential, and unclassified
 - MLS policy
 1. **Write**: A process can only write to segments of the same or higher-privileged secrecy levels
 2. **Read**: A process can only read from segments of the same or lower-privileged secrecy levels
 3. **Read/Write**: To read and write to a segment/directory the secrecy levels of the process and the segment/directory have to be the same

Influence on today's operating systems

- Ken Thompson and Dennis Ritchie were hired by Bell in the 1960s
- After Bell dropped out of the project, Thompson and Ritchie led a team to create **UNIX**
- Both won Turing Awards in 1983
- Commands like “ls”, “pwd” and “mail” were originally created in Multics

- The Multics project introduced many operating system concepts that are widely used in operating systems today

- Segmented and virtual memory ✓
- Shared memory multiprocessors ✓
- Hierarchical file systems ✓
- Online reconfiguration ✗
- Reference monitor ✓
- Protection rings ✓
- Multilevel security policies ✗
- Dynamic linking ✓

Windows reboots to configure new Software

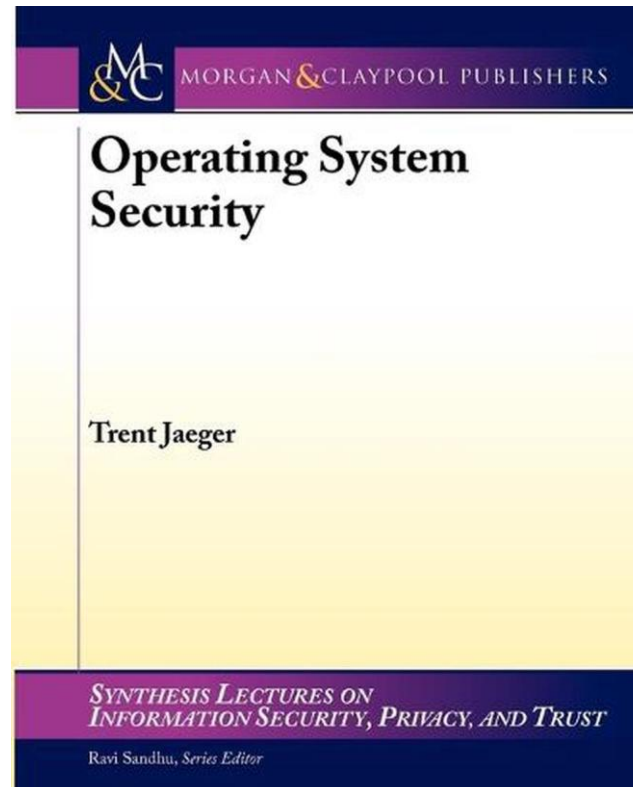
Modern systems only use 2 ring levels

- Multics was the first modern operating system
- Many important secure operating system concepts were pioneered in Multics, such as reference monitor, protection rings, etc.
- Multics realizes the reference monitor principles **complete mediation** and **tamperproof**. However, Multics TCB is too large to be formally verified
- Subsequent work in security kernels aimed to reduce the TCB complexity
 - A security kernel is defined as the combination of hardware and software that implements the reference monitor
 - The design includes hardware mechanisms leveraged by a minimal trusted computing base (TCB)
 - Security Kernel projects especially focused on **verifiability**
 - Therefore the TCB has to be as minimal as possible to allow reasonable verification

- A. Bensoussan, C.T. Clingen, and R.C. Daley:
“The Multics Virtual Memory: Concepts and Design”
<http://www.multicians.org/multics-vm.html>
- M. Gasser: “Building a Secure Computer System”
https://www.researchgate.net/publication/242363259_Building_a_Secure_Computer_System
- Technical paper of Honeywell: “The Multics Virtual Memory”
http://www.bitsavers.org/pdf/honeywell/multics/haley/AG95_part2_Jun72.pdf
- The Multicians web site presents the story and features of Multics:
<http://www.multicians.org/>
- J.H. Saltzer and M.D. Schroeder: “A Hardware Architecture for Implementing Protection Rings”
<http://www.multicians.org/protection.html>

Book Reference on OS Security including Multics

- T. Jaeger: "Operating System Security"
Morgan & Claypool Publishers, 2008



Vorlesung Cybersicherheit, Sommersemester 2025

Vielen Dank für Ihre Aufmerksamkeit

Prof. Dr.-Ing. Lucas Vincenzo Davi
Fakultät für Informatik
Arbeitsgruppe Systemsicherheit
Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi